Video Transcript

Video 3: Gradient Descent and the Line of Best Fit (11:16)

We're going to look at linear regression. Now this is built into the libraries, but here we're going to do it by hand first to understand how we can solve this problem. So, typically we have an independent variable X that we're making measurements at, and these are all independent measurements, that one measurement doesn't depend on the other. So, that's our assumption. And we have our dependent variable Y. And what we need to do is we're going to build a model that is in some sense a best fit. Given an independent variable, we can then predict the dependent variable.

So, for example, at this position out here, given this X, we would then be able to create a Y. Let's see how we do this in two dimensions. First so, just with X and Y, later on, we have many independent variables. We can do linear regression with many features that affect the dependent variable. So, we see that we can develop a model which we call ordinary linear regression where we have the equation Y equals w.X plus some error. And why we introduce the error is because when we have more than two points as we do here, it may be impossible to have zero error that this line will be a best fit line where we minimize the error according to some metric. So, we can write this equation out in this form in two dimensions. The X's are both the X coordinates, but also to represent the intercept we put a row of ones and so that's our matrix. And our predictive model will be w.X and that will give us our Y's. So, we can generate any Y given w's and the X's, we can generate the Y's. So, we have a predictive model. This can be solved analytically for smaller problems, but in general, in machine learning, we'll be using gradient descent. Now to do that, we must decide what we mean by error.

So, here we've got some plus errors, and here we have some minus errors they could cancel out. So, typically what we do is we'll square these differences. So, we'll take the difference between the actual Y coordinate and our predictive model. So, w.X, we'll square that and that will be our error term. Now, what does that error term look like? Well, because it's a squared, it looks like this in w0, so w0 along the bottom and w1 up. As w varies, we'll have an error that looks like this. So, we'll have some starting point here. We're starting at w0 is 0 and w1 is 0 . And then,

© MIT xPRO® 2024. All rights reserved.

we're trying to find out what w's give us the minimum error. So, we take the gradient so that'll be at right angles to these contour lines and we step down. And now here, there are other contour lines I'm not plotting, but it eventually finds a solution. Now, we hope it's the best solution, but we'll need an algorithm to find it. So, here's our algorithm that to calculate the gradient of the error with respect to w. So, both w's, w0 and w1, we can take the derivative. So, it was a squared.

So, with the two will come down in front, then we'll have Y minus w.X, and then we need to because it's with dW, we'll get another X outside. So, this will be our equation for the gradient. And now what we do is we'll change the w's, according to that gradient, we'll go in the opposite direction. So, if the gradient happens to be out, we want to go down and make it less, so we make it a minus. So, we'll take a delta gradient. And what we'll do to get that is we'll multiply the gradient by what we call a learning rate. It'll be a number less than one usually. So, our new w's will be whatever the old w is, minus some learning rate times the gradient. Okay, let's take a look at the code. We saw how to set up the X matrix. So, here we're just going to generate some points as a test case. And what I'm going to do is then I generate. So, these are the X's. Then I generate a number of ones, a row of ones, and stack them up. We have the ones first, then the X's. Now, I'm choosing my w to be 0, 0. So, here we are and this was what my Y would be. Everything would be 0. If w0 is 0 and w1 is 0, there's no slope and the intercept is 0. So, that's going to be my starting point. But what I'm going to do is I'm going to add 1 to the Y's. So, now this is my solution that I'm looking for. I'm going to be starting down here with this guess, but then iterating towards this solution. So, here's the code in action. So, it starts here, the first iteration it gets to this point, so it increases the w0, so that's the intercept, and it increases the slope.

You see here that it begins to overshoot with the slope and so it corrects that and eventually it gets the correct solution. Okay, so here's the actual code. Here I'm getting the gradient descent, so we'll go for a max number of iterations. We're calculating the change in w, so here's the learning rate, and here's the gradient function. So, we pass in w, and xy is a combination of X and Y. So, we're just putting it into a tuple, putting the X and the Y, and we'll just pass it around like that and then pull it out. So, here we see, when we're calculating the gradient, we take the xy and we separate the two. So, it's just a convenience. Then we get a predicted Y and we get the difference y - ypredicted. And remember, we need to multiply that by X. So, to get the

© MIT xPRO® 2024. All rights reserved.

gradient, the x is a vector, the difference is a scalar. So, the gradient will be, it'll give us a one-dimensional vector with two entries, the gradient of w0 and w1. And so, this is the mean square error calculation. And here's our model calculation, just w.X. So, here we're going to set a number of iterations and call it, loop over the iterations, and eventually we can plot how they did the error here. So, this is the cost or error function, loss function. So, it decreases as we iterate. And here, it's nearly zero. We've basically solved it. So, at the beginning we can take big steps, and as we get close to the solution, we need to take smaller steps and we'll talk about that later. So, let's just see this running. So, let me execute this, then this is the setup. So, I'm just testing against three points here, and this is the solution.

Now, I'm going to do some plotting, so we'll come back and look at how the plotting is done, but it's fairly straightforward. And so here, now let me show you these plots, they're running. And what we see is we're doing a different number of iterations and we're choosing here different learning rates. So here this is 0.001 and it finds a quite a good solution in 200 steps. In 50 iterations, it's a little bit out. You can see here, it should be flat and it's not quite. And then if we just do 20 iterations, it doesn't it doesn't converge yet. So, we need to play around with the learning rate and the number of iterations, those are the things that we can control here. I'm just showing you how it converges for, we're taking 200 iterations for that one. So, the best model minimizes the least square error. And, this will give us our solution. Okay, so why don't you exercise that code, make sure you can understand how it converges, and then you'll see I've added some other code here which allows you to test out how it converges. We'll see the little plot out here. In this one, it doesn't get to the final solution. However, when the learning rate is 0.5, it does. It gets very close here, so you can see how it goes down those contours towards the solution, and the solution is in the middle here. Okay, so gradient descent, it's used in machine learning, in neural networks, and in many other places. Okay, bye for now.

© MIT xPRO® 2024. All rights reserved.